

5교시: Compose로 전체 서비스 실행

Week 3 Day 1 · Lesson 5 실습 앱 다운로드 및 실행 준비

MSA 데모 실습을 위한 프로젝트를 내려받고, 실행 전 준비를 완료합니다.

폴더 구조 (week3/day1/labs/msa-demo)

```

msa-demo/
├── README.md
│   └── 프로젝트 개요 및 실행 안내
├── compose.yaml
│   └── 모든 서비스 정의 (Docker Compose)
├── .env.example
│   └── 환경 변수 예시 (복사하여 .env 생성)
├── frontend/
│   ├── 프론트엔드 (Nginx + static)
│   └── index.html 샘플 화면
├── api/
│   ├── 백엔드 API (Node.js + Express)
│   ├── Dockerfile
│   └── server.js
├── worker/
│   ├── 백그라운드 워커 (Node.js)
│   ├── Dockerfile
│   └── worker.js
└── db/
    ├── 데이터베이스 초기화 스크립트
    └── init/
        ├── 01-init.sql
        └── 02-seed.sql
  
```

실행 준비 (순서대로 진행)



명령어 예시

```

# 1. 저장소 복제
git clone <repository-url>

# 2. 폴더 이동
cd week3/day1/labs/msa-demo

# 3. 환경 파일 생성
cp .env.example .env

# 4. Compose 설정 검증
docker compose config
  
```

.env.example (예시)

```

# 데이터베이스
DB_USER=appuser
DB_PASSWORD=apppass
DB_NAME=msademo

# 애플리케이션
API_PORT=3000
WORKER_CONCURRENCY=2
QUEUE_NAME=jobs

# 보안 (예시)
JWT_SECRET=change_me
ADMIN_TOKEN=change_me_too
  
```

secret 주의

- .env 파일은 절대 Git에 커밋하지 마세요.
- 실제 운영 비밀번호, 토큰은 안전한 값으로 변경하세요.
- .env.example 은 예시일 뿐, 실제 값이 아닙니다.

.env → 로컬 전용 (Git 제외)
.env.example → 공유 가능

config 검증 (예상 출력 예시)

```

$ docker compose config
name: msa-demo
services:
  api:
    image: node:20-alpine
    ports:
      - "3000:3000"
  worker:
    image: node:20-alpine
    db:
      image: postgres:15-alpine
      volumes:
        - ../db/init:/docker-entrypoint-initdb.d:ro
  networks:
    default:
      driver: bridge
  
```

정상 출력 확인 포인트

- 모든 서비스(api, worker, db)가 정의되어 있는가?
- 포트 매핑(3000:3000 등)이 올바른가?
- 볼륨 경로(db/init)가 올바른가?
- 네트워크 설정이 존재하는가?

위 항목이 보이면 실행 준비 OK!

빠른 참조

- compose.yaml
모든 서비스와 네트워크 정의
- .env.example
환경 변수 템플릿
- README.md
실습 개요 및 실행 가이드

수업 목표

- msa-demo를 docker compose up --build -d로 실행한다.
- 실행 직후 baseline을 수집한다.
- container running, health, HTTP response, logs를 구분한다.

실행 전 준비

```

cd week3/day1/labs/msa-demo
cp .env.example .env
  
```

.env.example은 수업에서 어떤 runtime config가 필요한지 보여주는 파일이다. secret을 실제 운영 값으로 넣는 파일이 아니다.

확인:

```
cat .env.example
```

확인할 값:

값	의미
POSTGRES_PASSWORD	local practice DB password
DB_HOST	API가 DB를 찾을 service name
DB_PORT	DB container internal port

실행

```
docker compose up --build -d
docker compose ps
```

예상 형태:

NAME	SERVICE	STATUS
msa-demo-frontend-1	frontend	Up
msa-demo-api-1	api	Up (health: starting 또는 healthy)
msa-demo-worker-1	worker	Up
msa-demo-db-1	db	Up (healthy)

처음에는 API health가 starting일 수 있다. healthcheck interval과 DB readiness 때문에 잠깐 기다릴 수 있다.

정상 baseline 수집

```
curl -s http://localhost:18083/api/status
curl -s http://localhost:18084/health
docker compose logs --tail=40 api
docker compose logs --tail=40 worker
```

/api/status 예상 key:

```
{
  "service": "api",
  "frontend_to_api": "ok",
  "database_reachable": true,
  "db_host": "db",
  "db_port": 5432
}
```

여기서 `database_reachable=true`가 중요하다. 단순히 JSON이 온 것이 아니라 API가 DB까지 연결 가능하다는 증거다.

브라우저 확인

브라우저에서 다음 주소를 연다.

```
http://localhost:18083
```

브라우저 화면이 뜨면 frontend 정적 파일은 정상이다. 하지만 화면만 보고 끝내면 안 된다. 화면에서 API 결과가 표시되는지, 또는 직접 `curl /api/status`를 통해 API와 DB 연결까지 확인해야 한다.

baseline 표

확인	명령	정상 기준
Compose 상태	<code>docker compose ps</code>	frontend/api/worker/db 모두 Up, db healthy
frontend 경유 API	<code>curl localhost:18083/api/status</code>	JSON 응답, <code>database_reachable=true</code>
API 직접 health	<code>curl localhost:18084/health</code>	200, <code>ready=true</code>
API 로그	<code>docker compose logs api</code>	request path와 request id
worker 로그	<code>docker compose logs worker</code>	API 호출 status 200

흔한 실패

증상	원인 후보	첫 확인
port is already allocated	18083 또는 18084 사용 중	<code>docker ps</code>
API health 503	DB 미준비 또는 DB_HOST 오류	<code>docker compose logs api db</code>
frontend 502	nginx가 api에 연결 실패	<code>docker compose logs frontend api</code>
worker error 반복	API URL 오류 또는 API 중지	<code>docker compose logs worker</code>

Cleanup 기준

실습 도중에는 바로 지우지 않는다. Day2에서도 같은 앱을 쓰기 때문이다.

정리할 때:

```
docker compose down
```

DB data까지 초기화할 때만:

```
docker compose down -v
```

Evidence Note

```
# W3D1S5 Baseline
- docker compose ps summary:
- frontend API response:
- api health:
- worker log evidence:
- cleanup decision:
```

핵심 포인트

MSA 실습에서 baseline은 선택이 아니다. 정상 baseline이 있어야 Day2에서 장애 전파와 부분 장애를 비교할 수 있다.